

Цифровые технологии в производстве

Практические занятия 3 и 4

Условия и циклы в C#

Содержание работы

Практическое занятие 3 посвящено выбору действий по условию, занятие 4 – повторению вычислений в цикле. Для работы нужны переменные, арифметические выражения и ввод с клавиатуры, изученные на предыдущих занятиях. Задания рассчитаны на две пары: по одной паре на каждую практическую работу.

Сначала приведены основные конструкции языка C# и небольшие примеры. Затем разобраны две законченные программы. В конце находятся 30 индивидуальных вариантов. Студент выполняет две задачи своего варианта: первую – на занятии 3, вторую – на занятии 4. Выполнять все варианты не требуется.

Работа выполняется в Visual Studio в консольном приложении C#. Примеры написаны для C# 7.3 и подходят для проекта «Консольное приложение (.NET Framework)». В современном проекте .NET можно использовать тот же код, целиком заменив содержимое Program.cs. Класс Program и метод Main оставляют в файле только в одном экземпляре.

В условиях используются расчёты размеров, времени, расхода материала и выпуска продукции. Тарифы, предельные значения и нормы заданы для учебных задач. Все величины вводятся в указанных единицах; перевод единиц выполняется до подстановки в формулу.

Разделы методических указаний

Краткие теоретические сведения	3
Разбор программы с ветвлением.....	17
Разбор программы с циклом.....	19
Порядок выполнения работ	21
Индивидуальные задания	22
Оформление отчёта	37
Контрольные вопросы.....	37

Краткие теоретические сведения

1 Логический тип и операции сравнения

Условие в программе – выражение, результатом которого является одно из двух логических значений: `true` (истина) или `false` (ложь). Для хранения такого результата используется тип `bool`. Например, переменная `isReady` может обозначать готовность оборудования, а `isEnough` – наличие нужного количества материала. Логические значения записываются без кавычек. Строка `"true"` является текстом и не заменяет значение `true`.

Операции сравнения сопоставляют два значения. Числа могут быть заданы непосредственно, храниться в переменных или получаться в результате вычисления. Тип результата сравнения всегда `bool`. Основные операции приведены в таблице.

Операция	Значение	Пример
<code>a > b</code>	Больше	<code>12 > 10 → true</code>
<code>a < b</code>	Меньше	<code>12 < 10 → false</code>
<code>a >= b</code>	Больше или равно	<code>10 >= 10 → true</code>
<code>a <= b</code>	Меньше или равно	<code>10 <= 10 → true</code>
<code>a == b</code>	Равно	<code>10 == 10 → true</code>
<code>a != b</code>	Не равно	<code>10 != 10 → false</code>

```
int stock = 12;
int required = 10;
bool isEnough = stock >= required;
bool isEqual = stock == required;
Console.WriteLine(isEnough);
Console.WriteLine(isEqual);
stock = 8;
isEnough = stock >= required;
Console.WriteLine(isEnough);
```

Сначала на складе имеется 12 заготовок, а требуется 10. Условие `stock >= required` истинно. Равенство `stock == required` ложно, поскольку числа различаются. После присваивания `stock = 8` условие проверяется заново и становится ложным. Программа выведет `True`, `False` и `False`. Прописная буква относится к представлению результата на консоли; в коде логические литералы пишутся со строчной.

Присваивание и сравнение имеют разное назначение. Запись `stock = 10` изменяет количество, а `stock == 10` проверяет его. Замена двух знаков равенства одним не является сокращённой формой проверки. В условии `if` нельзя использовать целое число вместо `bool`: запись `if (stock)` для переменной `int` недопустима.

Включение границы зависит от знака сравнения. Условие `temperature < 80` не включает значение 80, а `temperature <= 80` включает. Если в условии задачи указано «не более», применяется `<=`; для «не менее» – `>=`. При проверке размеров границы записывают в тех же единицах, что и измеренную величину. Результаты вычислений с `double` могут быть приближёнными, поэтому условие годности обычно задают допустимым интервалом, а не точным равенством номиналу.

2 Логические операции и составные условия

Несколько проверок объединяют логическими операциями. Оператор `&&` означает «И»: результат истинен, когда истинны обе части. Оператор `||` означает «ИЛИ»: достаточно истинности хотя бы одной части. Оператор `!` означает отрицание и меняет `true` на `false`, а `false` на `true`.

A	B	A && B	A B
false	false	false	false
false	true	false	true
true	false	false	true
true	true	true	true

Для проверки принадлежности интервалу нужны две границы. Пусть допустимый диаметр находится от 39,95 до 40,05 мм включительно. Тогда размер должен быть одновременно не меньше нижней границы и не больше верхней. Выход за этот интервал означает, что размер либо меньше нижней границы, либо больше верхней.

```
double diameterMm = 40.02;
double lowerMm = 39.95;
double upperMm = 40.05;
bool inRange = diameterMm >= lowerMm
    && diameterMm <= upperMm;
bool outside = diameterMm < lowerMm
    || diameterMm > upperMm;
Console.WriteLine(inRange);
Console.WriteLine(outside);
Console.WriteLine(!inRange);
```

При размере 40,02 мм `inRange` равно `true`, а `outside` и `!inRange` равны `false`. Значения 39,95 и 40,05 также входят в интервал. Цепочка `lowerMm <= diameterMm <= upperMm` в C# недопустима: каждое сравнение записывается отдельно, затем результаты объединяются через `&&`.

Операции `&&` и `||` вычисляют левую часть первой. Для `&&` правая часть не вычисляется, если слева уже получено `false`. Для `||` правая часть не вычисляется, если слева `true`. Такое сокращённое вычисление позволяет сначала проверить допустимость дальнейшего действия.

```

int partCount = 25;
int boxCapacity = 0;
bool packedExactly = boxCapacity > 0
    && partCount % boxCapacity == 0;
Console.WriteLine(packedExactly);
boxCapacity = 5;
packedExactly = boxCapacity > 0
    && partCount % boxCapacity == 0;
Console.WriteLine(packedExactly);

```

При нулевой вместимости первая проверка ложна, поэтому остаток от деления на ноль не вычисляется. После изменения вместимости на 5 обе части истинны. Порядок здесь нельзя менять: деление должно следовать после проверки делителя. При сочетании разных логических операций сначала выполняется `!`, затем `&&`, затем `||`. Скобки позволяют явно задать нужную группировку, например `(a || b) && c`.

Отрицание составного условия записывают со скобками: `!(a && b)`. Для попадания в интервал требуется одновременное выполнение двух сравнений; отрицание такого условия означает нарушение хотя бы одной из границ.

3 Условный оператор if–else

Условный оператор выбирает действие в зависимости от логического выражения. В записи `if` (условие) условие заключается в круглые скобки. Если оно истинно, выполняется блок после `if`. Если ложно, этот блок пропускается. Необязательная часть `else` задаёт действие для ложного условия.

```

int stock = 20;
int required = 12;
if (stock >= required)
{
    stock -= required;
    Console.WriteLine("Материал выдан.");
    Console.WriteLine($"Остаток: {stock} шт.");
}
else
{
    Console.WriteLine("Материала недостаточно.");
}
Console.WriteLine("Проверка завершена.");

```

Для заданных значений выполняется первый блок. Из имеющихся 20 единиц вычитаются 12, поэтому остаток равен 8. Сообщение о недостатке не выводится. Если первоначально `stock` равно 7, первый блок будет пропущен: количество не уменьшится, а программа сообщит, что материала недостаточно. Строка после всей конструкции выполняется в обоих случаях.

Из двух ветвей `if–else` при одном прохождении выполняется только одна. Условие проверяется перед входом в выбранный блок. Если внутри него меняется переменная, использованная в условии, выбор ветви от этого не пересматривается. В примере после выдачи `stock` становится меньше `required`, но программа не переходит из блока `if` в `else`.

Фигурные скобки объединяют несколько инструкций в один блок. Для единственной инструкции язык допускает запись без них, однако в учебных программах скобки следует сохранять: при добавлении ещё одной строки границы ветви останутся ясными. Отступы помогают читать текст, но не определяют принадлежность инструкции к блоку.

После круглой скобки условия точку с запятой не ставят. Запись `if (stock >= required);` создаёт пустую инструкцию, а следующий блок уже не будет зависеть от проверки. Точка с запятой ставится после присваиваний и вызовов методов внутри блока.

Если при ложном условии ничего делать не требуется, часть `else` не записывается. Например, `if (temperature > limit)` может только вывести сообщение о превышении. Для нескольких независимых сообщений допустимы несколько отдельных `if`. Если же требуется выбрать один из взаимоисключающих результатов, используется одна конструкция с ветвями.

4 Несколько вариантов: `else if`

Цепочка `if–else if–else` применяется, когда требуется выбрать один результат из нескольких. Условия проверяются сверху вниз. Выполняется первая ветвь, условие которой истинно; остальные условия цепочки уже не рассматриваются. Последняя часть `else` выполняется, если не подошла ни одна из проверок.

Пусть учебная оценка качества выражена целым числом от 0 до 100. Значения от 90 относят к первой категории, от 70 до 89 – ко второй, а меньшие значения требуют доработки. Сначала проверяется допустимость введённых данных, затем категория.

```
int score = 83;
if (score < 0 || score > 100)
{
    Console.WriteLine("Недопустимое число баллов.");
}
else if (score >= 90)
{
    Console.WriteLine("Первая категория.");
}
else if (score >= 70)
{
    Console.WriteLine("Вторая категория.");
}
else
{
    Console.WriteLine("Требуется доработка.");
}
```

Для 83 первая проверка ложна: число находится в допустимом диапазоне. Проверка `score >= 90` также ложна. Условие `score >= 70` истинно, поэтому выводится вторая категория. Нижняя ветвь `else` не выполняется.

Порядок порогов определяет результат. Если раньше проверить `score >= 70`, значение 95 попадёт в эту ветвь. До проверки первой категории выполнение не дойдёт. При проверках вида «не меньше порога» более высокие пороги поэтому

располагают раньше. При проверках вида «меньше порога» обычно начинают с меньшего порога.

Вторая категория в примере не содержит явной проверки `score < 90`. Это ограничение уже известно: предыдущая ветвь была пропущена. После предварительной проверки диапазона также известно, что `score` находится от 0 до 100. Такая последовательность позволяет не повторять одинаковые сравнения в каждой ветви.

Баллы	Выполняемая ветвь
Меньше 0 или больше 100	Сообщение об ошибке
От 90 до 100	Первая категория
От 70 до 89	Вторая категория
От 0 до 69	Доработка

Границы относятся ровно к одной категории. Значение 70 попадает во вторую, 90 – в первую. Несколько отдельных `if` здесь дали бы другой порядок: для 95 истинны обе проверки `>= 90` и `>= 70`, поэтому появились бы два сообщения. Цепочка `else if` исключает такое повторное присвоение результата.

5 Вложенные условия и оператор ?:

Условный оператор может находиться внутри другой ветви. Внешняя проверка определяет, требуется ли выполнять внутреннюю. Такое построение удобно, если второй вопрос имеет смысл только после первого. Например, сведения о резерве рассматриваются только при наличии товара на складе.

```
int stock = 6;
bool reserved = true;
if (stock > 0)
{
    if (reserved)
    {
        Console.WriteLine("Товар находится в резерве.");
    }
    else
    {
        Console.WriteLine("Товар доступен для выдачи.");
    }
}
else
{
    Console.WriteLine("Товара нет на складе.");
}
```

В примере `stock` положительно, поэтому выполняется внешний блок. Внутри него проверяется `reserved`. Значение `true` приводит к сообщению о резерве. Если `stock` равно нулю, внутренняя проверка не выполняется вообще: состояние `reserved` не влияет на сообщение об отсутствии товара.

Каждая часть `else` относится к определённому `if`. При отсутствии фигурных скобок `else` связывается с ближайшим допустимым `if`, из-за чего код легко

прочитать неверно. Явные блоки показывают, какие действия относятся к наличию товара, а какие – к состоянию резерва. Для длинной последовательности равноправных проверок обычно удобнее `else if`, чем несколько уровней вложенности.

Когда требуется выбрать только одно из двух значений, можно использовать условный оператор `?:`. Его запись имеет вид `условие ? значение_при_true : значение_при_false`. Это выражение возвращает выбранное значение, которое затем присваивается переменной или передаётся в другой метод.

```
double diameterMm = 25.03;
double lowerMm = 24.95;
double upperMm = 25.05;
bool inRange = diameterMm >= lowerMm
    && diameterMm <= upperMm;
string result = inRange ? "Годная" : "Брак";
Console.WriteLine($"Размер: {diameterMm} мм");
Console.WriteLine(result);
```

Здесь условие `inRange` истинно, поэтому строка `result` получает значение «Годная». При выходе за границы выбирается «Брак». Оба результата имеют один тип `string`. В `C# 7.3` для таких выражений удобно использовать явно согласованные типы, не рассчитывая на возможности более новых версий языка.

Оператор `?:` не предназначен для записи длинных последовательностей действий. Если в каждой ветви нужно изменить несколько переменных и вывести сообщения, обычный `if-else` читается проще. Вложенные условные выражения также быстро затрудняют чтение: сокращение числа строк не должно скрывать порядок выбора.

6 Выбор по значению: `switch`

Оператор `switch` применяется для выбора по отдельным значениям: номеру операции, коду материала, пункту меню. В круглых скобках указывается проверяемое выражение. Метка `case` содержит значение, при совпадении с которым выполняется соответствующая группа инструкций.

```
int operation = 2;
string operationName;
switch (operation)
{
    case 1:
        operationName = "Отрезка";
        break;
    case 2:
        operationName = "Сверление";
        break;
    case 3:
        operationName = "Шлифование";
        break;
    default:
        operationName = "Неизвестная операция";
        break;
}
Console.WriteLine(operationName);
```

Значение `operation` равно 2, поэтому выбирается `case 2` и переменная `operationName` получает строку «Сверление». Оператор `break` завершает `switch`. Выполнение продолжается со строки `Console.WriteLine` после закрывающей фигурной скобки.

Ветка `default` предназначена для значения, которое не совпало ни с одним `case`. В примере она позволяет обработать, например, код 8. Эта ветка необязательна, но без неё неизвестный код не вызовет ни одного из перечисленных действий. Если после `switch` используется переменная, она должна получить значение на любом допустимом пути выполнения.

В C# нельзя неявно перейти из одной непустой ветки `case` в следующую. В приведённой форме каждая ветка завершается `break`. Пропуск завершающего перехода приводит к ошибке компиляции, а не к последовательному выполнению остальных вариантов. В других задачах ветку может завершать, например, `return`, если требуется выйти из метода.

Несколько меток могут относиться к одной группе инструкций. Например, `case 1:` и `case 2:` можно записать подряд перед общим блоком для двух кодов. Между такими пустыми метками `break` не требуется. При этом одинаковые значения `case` в одном `switch` повторять нельзя.

В рассматриваемых программах `case` содержит целочисленную константу, символ или строку. Для диапазонов температуры, размера или баллов проще использовать `if-else if`. Запись `case > 100` не относится к применяемому здесь синтаксису C# 7.3. Если сравниваются строки, регистр учитывается: «сталь» и «Сталь» являются разными значениями.

Оператор `switch` вычисляет проверяемое выражение и выбирает подходящую ветвь. Он не повторяет действия автоматически. Для повторного показа меню или обработки следующего кода понадобится цикл, внутри которого будет размещён `switch`.

7 Цикл `for`

Цикл повторяет группу инструкций. Одно выполнение тела цикла называется итерацией. Оператор `for` удобен, когда известен диапазон номеров или требуемое количество повторений. В его заголовке задаются начальное значение счётчика, условие продолжения и изменение счётчика.

```
int batchCount = 5;
double pieceMassKg = 2.4;
for (int number = 1; number <= batchCount; number++)
{
    double totalMassKg = number * pieceMassKg;
    Console.WriteLine($"Деталей: {number}");
    Console.WriteLine($"Масса: {totalMassKg:F1} кг");
}
Console.WriteLine("Расчёт завершён.");
```

Начальная инструкция `int number = 1` выполняется один раз. Затем проверяется `number <= batchCount`. Если условие истинно, вычисляется масса

указанного количества деталей и выводятся результаты. После тела выполняется `number++`, затем условие проверяется снова.

Счётчик последовательно принимает значения 1, 2, 3, 4 и 5. Соответствующие массы составляют 2,4; 4,8; 7,2; 9,6 и 12,0 кг. После пятой итерации `number` увеличивается до 6. Условие становится ложным, и программа переходит к сообщению после цикла. Если `batchCount` изначально равно нулю, тело не выполнится ни разу.

Три части заголовка разделяются точками с запятой. После закрывающей круглой скобки заголовка точка с запятой не ставится: далее начинается тело в фигурных скобках. Переменная, объявленная в начальной части `for`, доступна внутри заголовка и тела этого цикла. За его пределами это имя уже недоступно.

Шаг не обязан равняться единице. Запись `number += 2` перебирает числа через одно. Для убывающей последовательности начальное значение задают сверху, используют условие вида `number >= 1` и уменьшают счётчик. Направление шага должно позволять условию стать ложным.

При переборе N значений распространены две формы: от 1 до N включительно и от 0 до $N - 1$. Первая записывается как `i = 1; i <= N; i++`, вторая – `i = 0; i < N; i++`. Обе дают N итераций при положительном N . Смещение начального значения одной формы и границы другой приводит к лишнему или пропущенному повторению.

8 Цикл while

Оператор `while` выполняет тело, пока логическое условие истинно. Проверка производится перед каждой итерацией, в том числе перед первой. Поэтому тело может не выполниться ни разу. Такой цикл удобен, когда количество повторений определяется состоянием данных.

В следующем примере из остатка партии последовательно выделяются полные коробки. Вместимость считается положительной и равной пяти. Пока деталей достаточно для очередной полной коробки, остаток уменьшается на пять, а число заполненных коробок увеличивается на единицу.

```
int remaining = 23;
int boxCapacity = 5;
int fullBoxes = 0;
while (remaining >= boxCapacity)
{
    remaining -= boxCapacity;
    fullBoxes++;
}
Console.WriteLine($"Полных коробок: {fullBoxes}");
Console.WriteLine($"Осталось деталей: {remaining}");
```

Перед первой итерацией `remaining` равно 23. После последовательных вычитаний оно принимает значения 18, 13, 8 и 3. Проверка `3 >= 5` ложна, поэтому цикл заканчивается. Полных коробок получается четыре, остаток равен трём деталям. Если исходный остаток равен 4, условие сразу ложно и `fullBoxes` остаётся нулём.

Перед итерацией	После вычитания	Полных коробок
23	18	1
18	13	2
13	8	3
8	3	4

Изменение `remaining` находится внутри тела. Если его удалить, условие `23 >= 5` останется истинным при каждой проверке, и цикл не завершится обычным путём. При написании `while` нужно установить, какая инструкция меняет состояние и за счёт чего наступит завершение.

Знак `>=` в этом примере выбран намеренно. При остатке ровно 5 ещё можно заполнить одну коробку. Условие `remaining > boxCapacity` пропустило бы этот случай. Если вместимость поступает от пользователя, до входа в цикл требуется проверить `boxCapacity > 0`: вычитание нуля не изменит остаток, а отрицательная вместимость увеличит его.

Инициализация переменных располагается перед `while`. Она не повторяется вместе с телом. Число коробок также объявлено до цикла, поэтому после завершения сохраняет накопленный результат. Если присваивать `fullBoxes = 0` внутри каждой итерации, предыдущий счёт будет теряться. В отличие от `for`, оператор `while` не содержит отдельной части для шага: все изменения записываются в теле.

9 Цикл `do-while`

Цикл `do-while` проверяет условие после выполнения тела. Сначала выполняются инструкции между `do` и закрывающей фигурной скобкой, затем вычисляется выражение после `while`. Если оно истинно, тело повторяется. Поэтому `do-while` всегда выполняет хотя бы одну итерацию.

Эта форма подходит для действий, которые должны произойти до решения о повторении. Например, программа сначала принимает название детали и выводит его, затем предлагает продолжить ввод. Переменная `answer` объявляется перед циклом, чтобы она была доступна в заключительном условии.

```
string answer;
do
{
    Console.Write("Название детали: ");
    string partName = Console.ReadLine();
    Console.WriteLine($"Принято: {partName}");
    Console.Write("Продолжить ввод? (да/нет): ");
    answer = Console.ReadLine();
}
while (answer == "да");
```

После первого названия пользователь вводит ответ. При строке «да» начинается следующая итерация; при другой строке цикл заканчивается. Сравнение строк учитывает регистр, поэтому в этом простом примере «Да» не совпадает с «да». Пустая строка также завершит повторение. В программе с другим порядком ввода правило обработки ответа можно изменить явно.

В отличие от `while` и `for`, после заключительного условия `do-while` ставится точка с запятой. Она завершает всю конструкцию. Само тело по-прежнему выделяется фигурными скобками. Переменная `partName` создаётся внутри тела и используется в этой итерации; после цикла она недоступна.

```
int number = 10;
do
{
    Console.WriteLine(number);
    number++;
}
while (number < 5);
Console.WriteLine($"После цикла: {number}");
```

Во втором фрагменте начальное число уже не удовлетворяет условию `number < 5`. Тем не менее программа выводит 10 и увеличивает `number` до 11. Только после этого выполняется проверка и цикл завершается. Если те же действия поместить в `while (number < 5)`, тело не выполнится ни разу.

Выбор между `while` и `do-while` определяется требуемым порядком. Если действие разрешено только после проверки, подходит `while`. Если сначала необходимо получить данные или выполнить первую попытку, а затем решать о повторении, подходит `do-while`. Наличие цикла не отменяет проверку введённых значений: считывание числа и сообщение об ошибке задаются отдельными инструкциями.

10 Досрочное завершение и пропуск итерации

Оператор `break` завершает ближайший цикл, внутри которого он выполнен. Программа продолжает работу с первой инструкции после этого цикла. Оператор `continue` не завершает весь цикл: он пропускает оставшуюся часть текущей итерации и передаёт управление к следующей проверке или шагу.

```
for (int number = 1; number <= 10; number++)
{
    if (number > 7)
    {
        break;
    }
    if (number % 3 == 0)
    {
        continue;
    }
    Console.WriteLine(number);
}
Console.WriteLine("Перебор закончен.");
```

В примере выводятся числа 1, 2, 4, 5 и 7. Значения 3 и 6 пропускаются, поскольку остаток от их деления на 3 равен нулю. При `number = 8` выполняется

break, поэтому значения 8, 9 и 10 уже не рассматриваются. После завершения выводится последнее сообщение.

Расположение проверки влияет на результат. Если Console.WriteLine поставить перед условием break, число 8 успеет попасть в вывод. Аналогично continue пропускает только те инструкции, которые стоят после него в теле; уже выполненные действия не отменяются.

В цикле for после continue выполняется изменение счётчика из заголовка, затем проверяется условие продолжения. В while отдельного шага нет. Поэтому переход continue может случайно пропустить изменение переменной, необходимое для завершения. В следующем фрагменте увеличение number расположено до проверки и происходит на каждой итерации.

```
int number = 0;
while (number < 5)
{
    number++;
    if (number == 3)
    {
        continue;
    }
    Console.WriteLine(number);
}
```

Здесь выводятся 1, 2, 4 и 5. Значение 3 пропущено, но цикл продолжает изменять счётчик и завершается. Если увеличение перенести после continue, при достижении трёх оно может больше не выполниться.

При вложенных циклах break прекращает только ближайший внутренний цикл. Для switch этот оператор завершает именно switch, если тот расположен ближе, чем внешний цикл. Оператор return имеет другое назначение: он выходит из всего метода. Поэтому break и return нельзя считать взаимозаменяемыми командами завершения программы.

Досрочный выход применяют при обнаружении искомого значения или признака остановки. Continue используют, когда отдельное значение нужно исключить из дальнейшей обработки, но остальные значения должны быть рассмотрены.

11 Счётчик, сумма, среднее, минимум и максимум

При обработке последовательности часто требуется накопить общий результат. Счётчик хранит количество принятых значений, а сумма – результат их сложения. Обе переменные объявляются до цикла и получают начальное значение ноль. Внутри цикла сумма увеличивается на очередное значение, счётчик – на единицу.

```

int count = 0;
double sum = 0.0;
for (int number = 1; number <= 4; number++)
{
    double sizeMm = 10.0 + number;
    sum += sizeMm;
    count++;
}
double averageMm = sum / count;
Console.WriteLine($"Количество: {count}");
Console.WriteLine($"Сумма: {sum:F1} мм");
Console.WriteLine($"Среднее: {averageMm:F1} мм");

```

В примере последовательно обрабатываются размеры 11, 12, 13 и 14 мм. Сумма равна 50 мм, количество – четырём, среднее – 12,5 мм. Выражение `10.0 + number` здесь только задаёт последовательность для разбора алгоритма. В программе с измерениями его заменяют чтением очередного размера.

Если учитываются не все значения, счётчик и сумма должны изменяться в одном и том же условном блоке. Например, при усреднении только положительных измерений отрицательные значения не добавляются ни в сумму, ни в количество. Перед делением проверяют `count > 0`. Для пустой последовательности среднее, минимум и максимум не определены.

Минимум и максимум обычно инициализируются первым принятым значением. Начальный ноль подходит не всегда: для набора положительных размеров он ошибочно останется минимумом. После инициализации каждое следующее значение сравнивают с уже найденными границами.

```

double firstMm = 11.0;
double minMm = firstMm;
double maxMm = firstMm;
for (int number = 2; number <= 4; number++)
{
    double sizeMm = 10.0 + number;
    if (sizeMm < minMm)
    {
        minMm = sizeMm;
    }
    if (sizeMm > maxMm)
    {
        maxMm = sizeMm;
    }
}
Console.WriteLine($"Минимум: {minMm} мм");
Console.WriteLine($"Максимум: {maxMm} мм");

```

Первый размер 11 мм уже учтён, поэтому цикл начинается со второго. Значения 12, 13 и 14 последовательно увеличивают максимум; минимум остаётся равным 11. При равенстве очередного значения найденной границе обновление не требуется.

Среднее вычисляют после накопления всей суммы. Если делить сумму на количество внутри каждой итерации и сохранять частное обратно в сумму, её исходный смысл будет потерян. Форматирование результата также выполняют

при выводе: промежуточные размеры и накопленную сумму без необходимости не округляют.

12 Проверка ввода и размещение фрагментов в программе

Приведённые фрагменты предназначены для консольного приложения. Их инструкции размещают внутри метода Main. Директивы using записывают в начале файла, за пределами класса. Для Console требуется System, для настройки формата чисел – System.Globalization. Общая структура файла для C# 7.3 имеет следующий вид.

```
using System;
using System.Globalization;

class Program
{
    static void Main()
    {
        // Здесь размещается программный фрагмент.
    }
}
```

При переносе примера в эту структуру не нужно повторно создавать Main внутри Main. Имена локальных переменных в одном блоке не должны повторяться, поэтому самостоятельные примеры запускают отдельно либо согласуют их объявления.

Console.ReadLine возвращает строку. Метод TryParse пытается получить из неё число и сообщает результат через bool. Число передаётся в переменную после out. Неуспешное преобразование не следует заменять нулём: требуется сообщить об ошибке и повторить ввод.

В следующем фрагменте допустима длина больше нуля и не больше 1000 мм. Культура ru-RU задаётся явно; при вводе дробная часть отделяется запятой. В исходном коде числовые литералы по-прежнему содержат точку.

```
CultureInfo.CurrentCulture =
    new CultureInfo("ru-RU");
double lengthMm;
while (true)
{
    Console.WriteLine("Длина, мм (больше 0; не более 1000): ");
    string input = Console.ReadLine();
    if (input == null) { return; }
    bool isNumber = double.TryParse(
        input, NumberStyles.Float,
        CultureInfo.CurrentCulture, out lengthMm);
    if (isNumber && lengthMm > 0 && lengthMm <= 1000)
    {
        break;
    }
    Console.WriteLine("Нужно число больше 0 и не больше 1000.");
}
Console.WriteLine($"Принята длина: {lengthMm:F2} мм");
```

Строка «25,5» проходит обе проверки. Слово «длина» не преобразуется в число, а 0 и –5 не входят в заданный диапазон. В этих случаях цикл

повторяется. При верных данных `break` завершает цикл, и принятое значение используется далее. Если входной поток закончился и `ReadLine` вернул `null`, `return` завершает `Main`.

`NumberStyles.Float` разрешает обычную дробную запись, знак и экспоненту, но не разделители групп разрядов. Для целого количества применяется `int.TryParse`. Проверки числовой формы и диапазона выполняют отдельно по смыслу, даже если они объединены одним условием.

Для цен и величин с заданным десятичным шагом, например припуска 0,01 мм, подходит тип `decimal`. Его числовые литералы записывают с суффиксом `m`: 0.01m или 125.50m. Значения 0.1m и 0.2m складываются точно в 0.3m. Для ввода применяют `decimal.TryParse(input, out price)`, предварительно объявив `decimal price`; разделитель определяется текущей культурой. После успешного преобразования отдельно проверяют положительность и границы, указанные в условии. Смешивать `decimal` и `double` в одной формуле без явного преобразования нельзя.

Ограничения ввода берутся из задачи. Положительная длина, ненулевой делитель и количество повторений в допустимом диапазоне проверяются до вычисления или входа в основной цикл. Сообщение должно указывать конкретную причину повторного ввода и допустимые значения.

Разбор программы с ветвлением

За смену необходимо изготовить `plan` деталей. Фактически изготовлено `fact` деталей. Требуется найти процент выполнения плана и вывести одно из трёх сообщений: о недовыпуске, точном выполнении или выпуске сверх плана. План должен быть положительным целым числом, фактический выпуск – целым числом не меньше нуля.

Процент выполнения равен $100 \times \text{fact} / \text{plan}$. Число 100.0 в программе записано с точкой, чтобы деление выполнялось как вещественное. При плане 120 деталей и выпуске 114 деталей получаем 95 %. Недовыпуск составляет $120 - 114 = 6$ деталей.

Сначала проверяется ввод. Только после успешной проверки можно делить на `plan`. Затем вычисляется процент и выбирается сообщение. Ветви должны быть взаимоисключающими: одному набору исходных данных соответствует ровно одно заключение.

Условие	Действие
<code>fact < plan</code>	Вычислить <code>plan – fact</code> и вывести недовыпуск
<code>fact == plan</code>	Сообщить о точном выполнении
<code>fact > plan</code>	Вычислить <code>fact – plan</code> и вывести выпуск сверх плана

Ниже приведено полное содержимое `Program.cs`. В этой программе ошибочный ввод завершает расчёт: оператор `return` выходит из метода `Main`. Повторный запрос числа здесь не нужен, поскольку пример посвящён ветвлению.

```
using System;

class Program
{
    static void Main()
    {
        int plan;
        int fact;

        Console.Write("План выпуска, шт.: ");
        if (!int.TryParse(Console.ReadLine(), out plan)
            || plan <= 0)
        {
            Console.WriteLine("План должен быть положительным целым.");
            return;
        }

        Console.Write("Фактический выпуск, шт.: ");
        if (!int.TryParse(Console.ReadLine(), out fact)
            || fact < 0)
        {
            Console.WriteLine("Выпуск должен быть целым и неотрицательным.");
            return;
        }
    }
}
```

```

double percent = 100.0 * fact / plan;
Console.WriteLine("Выполнение плана: {0:F1} %", percent);

if (fact < plan)
{
    Console.WriteLine("Недовыпуск: {0} шт.", plan - fact);
}
else if (fact == plan)
{
    Console.WriteLine("План выполнен точно.");
}
else
{
    Console.WriteLine("Сверх плана: {0} шт.", fact - plan);
}
}
}

```

В условии с TryParse сначала проверяется, удалось ли получить целое число. Если преобразование не удалось, выражение после || не вычисляется. Если преобразование прошло успешно, проверяется допустимость значения. Поэтому запись 12,5 не принимается как количество деталей, а отрицательный выпуск отвергается, хотя является корректно записанным целым числом.

Проверим все три ветви. Для плана 120 и выпуска 114 программа должна показать 95,0 % и недовыпуск 6 деталей. При выпуске 120 получится 100,0 % и сообщение о точном выполнении. При выпуске 132 получится 110,0 % и 12 деталей сверх плана. Десятичный разделитель в выводе зависит от региональных настроек Windows.

План	Выпуск	Ожидаемый результат
120	114	95,0 %; недовыпуск 6 шт.
120	120	100,0 %; план выполнен точно
120	132	110,0 %; сверх плана 12 шт.
120	0	0,0 %; недовыпуск 120 шт.
0	—	Сообщение об ошибке плана; расчёт не выполняется

Не следует определять выполнение плана по округлённому проценту. Например, 999 деталей при плане 1000 – это недовыпуск, даже если вывести процент без десятичных знаков и увидеть 100 %. В программе сравниваются исходные целые количества, а формат F1 применяется только к выводу.

Разбор программы с циклом

Для нескольких операций записано фактическое время выполнения в целых секундах. Число операций count известно заранее и находится в пределах от 1 до 20. Продолжительность каждой операции – от 1 до 3600 с. Требуется найти общее время, среднее время, наибольшее время и количество операций длительностью больше 60 с.

Сохранять все введенные значения не требуется. После чтения очередного числа seconds его прибавляют к сумме. Если оно больше ранее найденного максимума, максимум заменяют. Если операция длилась больше 60 с, увеличивают отдельный счётчик. Затем переходят к следующему значению.

Переменная	Что хранит	Начальное значение
sum	Сумму введенных времён	0
maximum	Наибольшее введенное время	0, так как все времена положительны
longCount	Число операций дольше 60 с	0
i	Номер текущей операции	1

Цикл for удобен, поскольку число повторений известно. При $i = 1$ вводится первая операция, при $i = \text{count}$ – последняя. После увеличения i до $\text{count} + 1$ условие $i \leq \text{count}$ станет ложным. Среднее рассчитывается один раз, после цикла.

```
using System;

class Program
{
    static void Main()
    {
        int count;
        Console.Write("Число операций от 1 до 20: ");
        if (!int.TryParse(Console.ReadLine(), out count)
            || count < 1 || count > 20)
        {
            Console.WriteLine("Неверное число операций.");
            return;
        }

        int sum = 0;
        int maximum = 0;
        int longCount = 0;
```

```

for (int i = 1; i <= count; i++)
{
    int seconds;
    Console.Write("Время операции {0}, с: ", i);
    if (!int.TryParse(Console.ReadLine(), out seconds)
        || seconds < 1 || seconds > 3600)
    {
        Console.WriteLine("Введите целое время от 1 до 3600 с.");
        return;
    }

    sum += seconds;
    if (seconds > maximum)
        maximum = seconds;
    if (seconds > 60)
        longCount++;
}

double average = (double)sum / count;
Console.WriteLine("Всего: {0} с", sum);
Console.WriteLine("Среднее: {0:F2} с", average);
Console.WriteLine("Максимум: {0} с", maximum);
Console.WriteLine("Дольше 60 с: {0}", longCount);
}
}

```

Преобразование `(double)sum` необходимо для вещественного деления. Если написать `sum / count`, оба операнда будут целыми и дробная часть результата потеряется ещё до присваивания переменной `average`. Формат `F2` выведет два десятичных знака, но не восстановит потерянную дробную часть.

Проследим выполнение для четырёх операций: 45, 72, 60 и 83 с. В таблице указаны значения переменных после обработки очередного числа. Время 60 с не увеличивает `longCount`, потому что по условию учитываются операции строго дольше 60 с.

i	seconds	sum	maximum	longCount
1	45	45	45	0
2	72	117	72	1
3	60	177	72	1
4	83	260	83	2

Итог программы: общее время 260 с, среднее 65,00 с, максимум 83 с, операций дольше минуты – 2. Для проверки границы достаточно отдельно запустить программу с одной операцией длительностью 60 с: сумма, среднее и максимум равны 60, счётчик `longCount` равен нулю.

Начальное значение `maximum = 0` выбрано по условию задачи. Здесь все допустимые значения положительны. Для серии отклонений, среди которых могут быть отрицательные числа, такой способ уже не подходит: максимум нужно начать с первого измерения. Также нельзя обнулять `sum` и `longCount` внутри тела цикла – тогда на каждом шаге результаты предыдущих шагов будут теряться.

Порядок выполнения работ

Создайте отдельный консольный проект для каждой работы. В начале Program.cs укажите номер варианта и работы в комментарии. Переменные называйте по смыслу; обозначения из формул поясняйте в комментариях или приглашениях к вводу.

До написания кода выпишите исходные данные, единицы измерения и формулы. В задаче на ветвление перечислите возможные случаи. В задаче на цикл определите повторяемые действия, условие завершения и накапливаемые результаты.

В работе 3 используйте if, else if, else или switch. В работе 4 – for, while или do while с необходимыми условиями внутри цикла. Массивы, коллекции, LINQ и отдельные пользовательские методы здесь не требуются.

Переменные исходные данные вводите с клавиатуры. Заданные в условии постоянные можно записать в коде. Количества деталей, заказов и других объектов – целые числа. Если не указано иначе, число элементов N находится в пределах от 1 до 20.

Длины, массы, цены, скорости и нормы расхода должны быть положительными; остаток запаса и фактический выпуск могут быть нулевыми. Отклонения размеров могут быть отрицательными. Особые ограничения, включая допустимость нуля, указаны в вариантах. Для кодов проверяйте принадлежность заданному набору.

Проверяйте ввод через TryParse. При ошибке формата или диапазона сообщите её причину и завершите расчёт либо повторите запрос. Ошибочные значения не включайте в итоги. В приглашении к вводу дробного числа укажите принятый десятичный разделитель.

Не округляйте промежуточные результаты, если этого не требует условие. Для денежных величин и точных десятичных границ используйте decimal. Целые микроны и сотые доли миллиметра сравнивайте до перевода единиц. Тип double нужен, в частности, для расчётов с Math.PI.

Каждое выведенное число подпишите и снабдите единицей измерения. Если точность не задана, выводите вещественный результат с двумя знаками. Текст сообщения может отличаться от контрольного примера, но его смысл и числа должны совпадать.

Проверьте контрольный пример, затем другие ветви и границы условий. Для «не больше 100» нужны проверки 99, 100 и 101. Для цикла проверьте минимально допустимое число записей и более длинную серию; для отбора – отсутствие подходящих значений.

В контрольных примерах ввод записан сокращённо. «N = 3; значения 10, 20, 30» означает три последовательных ввода с отдельных строк. Разделяющие данные точки с запятой в консоль не вводятся. В записи «код/время» сначала вводится код, затем время.

Вариант 1 Проходы при точении

Практическая работа 3

С заготовки нужно снять припуск h на сторону. За один проход снимают не больше t . В этой задаче h и t заданы целым числом сотых долей миллиметра. Если h равен нулю, обработка не нужна. Если h не превышает t , достаточно одного прохода. В остальных случаях число проходов округляют вверх: $n = (h + t - 1) / t$ при целочисленном делении. Глубина последнего прохода равна $h - (n - 1)t$. При точном делении последний проход полный.

Ввести: h – припуск, сотые доли мм; t – предельная глубина прохода, сотые доли мм.

Вывести: число проходов; глубину последнего прохода в миллиметрах с двумя знаками. При $h = 0$ вывести сообщение «Обработка не нужна».

Контрольный пример. Вход: $h = 375$, $t = 120$. Результат: Проходов 4; последний проход 0,15 мм.

Практическая работа 4

После обработки записаны глубины отдельных проходов. Нужно сложить их и сравнить снятый припуск с заданным. Значения считываются по одному в цикле. Если сумма равна заданию, припуск снят полностью. При меньшей сумме вывести остаток, при большей – величину превышения. Все исходные глубины заданы в сотых долях миллиметра, поэтому сумма и сравнение выполняются с целыми числами. Перевод в миллиметры нужен только при выводе.

Ввести: h – заданный припуск; N – число выполненных проходов; затем N значений глубины.

Вывести: суммарный снятый припуск в миллиметрах и заключение; при расхождении – остаток или превышение с двумя знаками.

Контрольный пример. Вход: $h = 375$, $N = 4$; глубины 120, 120, 120, 15. Результат: Снято 3,75 мм; припуск снят полностью.

Вариант 2 Выбор ступени шпинделя

Практическая работа 3

Для учебного расчёта заданы три доступные частоты шпинделя: 500, 1000 и 1500 об/мин. По диаметру D и скорости резания v найти расчётную частоту $n = 1000v / (\pi D)$. Выбрать наибольшую доступную частоту, не превышающую расчётную. Если расчётная частота меньше 500 об/мин, подходящей ступени нет. При равенстве доступная ступень подходит. После выбора найти фактическую скорость $v_f = \pi D n / 1000$. Значение π взять из Math.PI; промежуточную частоту не округлять.

Ввести: D – диаметр, мм; v – заданная скорость, м/мин.

Вывести: расчётную частоту с одним знаком; выбранную ступень; фактическую скорость с двумя знаками. Если ступени нет, скорость не вычислять.

Контрольный пример. Вход: $D = 50$ мм, $v = 200$ м/мин. Результат: Расчётная частота 1273,2 об/мин; ступень 1000 об/мин; скорость 157,08 м/мин.

Практическая работа 4

При проверке шпинделя получено N измерений частоты. Номинальная частота n_0 одинакова для всей серии. С помощью цикла определить среднюю частоту и число измерений, отличающихся от n_0 больше чем на 5 %. Граница ровно 5 % считается допустимой. Для каждого измерения относительное отклонение равно $100 \times |n - n_0| / n_0$. Дополнительно найти наибольшее относительное отклонение за серию, независимо от его направления.

Ввести: n_0 – номинальная частота, об/мин; N ; затем N измеренных частот.

Вывести: среднюю частоту с двумя знаками; число превышений 5 %; наибольшее отклонение в процентах с двумя знаками.

Контрольный пример. Вход: $n_0 = 1000$, $N = 4$; частоты 980, 1000, 1020, 1100. Результат: Средняя 1025,00 об/мин; превышений 1; наибольшее отклонение 10,00 %.

Вариант 3 Машинное время операции

Практическая работа 3

Резец проходит длину обработки L и ещё 3 мм на врезание и перебег. При частоте n и подаче s машинное время одного прохода равно $T = (L + 3) / (ns)$. Результат получается в минутах. Разделить операции на три группы: до 2 минут включительно, больше 2, но не больше 5 минут, и больше 5 минут. Выбор группы выполнить цепочкой `if – else if – else`. Группу определять по рассчитанному времени до округления.

Ввести: L – длина обработки, мм; n – частота, об/мин; s – подача, мм/об.

Вывести: машинное время с тремя знаками и название группы по продолжительности.

Контрольный пример. Вход: $L = 420$, $n = 800$, $s = 0,25$. Результат: Время 2,115 мин; группа «Больше 2, но не больше 5 минут».

Практическая работа 4

За отведённое время F выполняется несколько операций. Их фактическую продолжительность вводят последовательно; ноль завершает ввод и не считается операцией. Записать цикл, который накопит общее время и число операций. После завершения сравнить сумму с F : определить оставшееся время либо перерасход. Если операции не введены, среднее время не вычислять и вывести сообщение об отсутствии данных. За один запуск вводится не больше двадцати операций.

Ввести: F – отведённое время, мин; затем времена операций в минутах, в конце 0.

Вывести: число операций; суммарное и среднее время с двумя знаками; остаток либо перерасход времени.

Контрольный пример. Вход: $F = 15$; времена 3,2; 2,8; 4; 0. Результат: Операций 3; всего 10,00 мин; среднее 3,33 мин; осталось 5,00 мин.

Вариант 4 Выбор станка и стоимость обработки

Практическая работа 3

На участке два условных станка. Станок А принимает заготовки диаметром до 100 мм и длиной до 300 мм, стоимость работы – 600 руб/ч. Для станка Б пределы составляют 200 и 600 мм, стоимость – 1000 руб/ч. Выбрать А, если оба размера подходят; иначе проверить Б. Равенство предельному размеру допускается. Если не подходит ни один станок, вывести соответствующее сообщение. Для выбранного станка стоимость обработки равна тарифу, умноженному на время в часах.

Ввести: D – диаметр, мм; L – длина, мм; T – время обработки, мин.

Вывести: выбранный станок и стоимость с двумя знаками. Для неподходящей заготовки стоимость не рассчитывать.

Контрольный пример. Вход: $D = 120$, $L = 250$, $T = 18$. Результат: Станок Б; стоимость 300,00 руб.

Практическая работа 4

Для расчёта загрузки задан список выполненных работ. Каждая запись содержит код станка и продолжительность. Код 1 соответствует станку А, код 2 – станку Б; тарифы взять из предыдущей задачи. В цикле отдельно суммировать время для каждого станка. Затем вычислить общую стоимость всех работ и определить, у какого станка время работы больше. При одинаковой продолжительности вывести «Загрузка одинакова». Переходы между работами и простои в расчёт не включаются.

Ввести: N – число работ; для каждой работы код станка и время, мин.

Вывести: загрузку А и Б в минутах; общую стоимость с двумя знаками; станок с большей загрузкой.

Контрольный пример. Вход: $N = 4$; пары код/время: 1/10, 2/15, 1/20, 2/30. Результат: А – 30 мин, Б – 45 мин; стоимость 1050,00 руб.; больше загружен Б.

Вариант 5 Проверка припуска заготовки

Практическая работа 3

Из заготовки диаметром D нужно получить деталь диаметром d . Припуск на сторону равен $z = (D - d) / 2$. Задан минимальный требуемый припуск $z_{\min}$. Если D меньше d , заготовка мала. Если диаметры равны, припуска нет. При положительном z меньше $z_{\min}$ припуск недостаточен; при z не меньше $z_{\min}$ – достаточен. Все четыре случая должны иметь отдельное сообщение. Размеры и припуски хранить в `decimal`.

Ввести: D и d – диаметры, мм; $z_{\min}$ – минимальный припуск на сторону, мм.

Вывести: расчётный припуск со знаком с тремя знаками и заключение о заготовке.

Контрольный пример. Вход: $D = 50$; $d = 48,8$; $z_{\min} = 0,5$. Результат: Припуск 0,600 мм; достаточен.

Практическая работа 4

Для одной детали диаметром d проверяют N заготовок разных диаметров. Подходящей считать заготовку с припуском на сторону от 0,5 до 2 мм включительно. Для каждого введённого D вычислить z по той же формуле в `decimal`. Подсчитать подходящие заготовки и сложить только их припуски. После серии найти средний припуск подходящих заготовок. Если таких заготовок нет, среднее не вычислять. Неподходящие заготовки не должны попадать ни в сумму, ни в знаменатель среднего.

Ввести: d – диаметр детали, мм; N ; затем N диаметров заготовок.

Вывести: число подходящих и неподходящих заготовок; средний припуск подходящих с тремя знаками либо сообщение об их отсутствии.

Контрольный пример. Вход: $d = 20$, $N = 5$; диаметры 20,8; 21; 22; 24; 24,2. Результат: Подходящих 3, неподходящих 2; средний припуск 1,167 мм.

Вариант 6 Слои печатной модели

Практическая работа 3

Высота модели H , первого слоя h_1 и остальных слоёв h заданы целыми микрометрами. В учебной модели последний слой допускается уменьшать до оставшейся высоты. Если H не превышает h_1 , модель состоит из одного слоя высотой H . Иначе после первого слоя остаётся $H - h_1$; число дополнительных слоёв равно $(H - h_1 + h - 1) / h$ при целочисленном делении. Если общее число слоёв $n > 1$, высота последнего равна $H - h_1 - (n - 2)h$.

Ввести: H – высота модели, мкм; h_1 – высота первого слоя, мкм; h – обычного слоя, мкм.

Вывести: общее число слоёв; фактическую высоту последнего слоя в миллиметрах с тремя знаками.

Контрольный пример. Вход: $H = 1050$, $h_1 = 300$, $h = 200$. Результат: Слоёв 5; последний слой 0,150 мм.

Практическая работа 4

При печати записано время выполнения каждого слоя. В начале принтер потратил 30 секунд на подготовку; это время учитывается один раз. С помощью цикла найти полную продолжительность и самый долгий слой. Если максимальное время встречается несколько раз, оставить номер первого такого слоя. Дополнительно подсчитать слои, печатавшиеся больше 12 секунд; ровно 12 секунд превышением не считается. Нумерация слоёв начинается с единицы. Исходные времена заданы целыми секундами.

Ввести: N – число слоёв; затем N значений времени печати слоя.

Вывести: полное время в минутах и секундах; номер и время самого долгого слоя; число слоёв дольше 12 секунд.

Контрольный пример. Вход: $N = 5$; времена 8, 12, 15, 10, 15 с. Результат: Всего 1 мин 30 с; самый долгий слой № 3, 15 с; дольше 12 с – 2 слоя.

Вариант 7 Расход филамента на партию

Практическая работа 3

Для одной детали требуется m граммов филамента. На подготовку и возможные потери к расходу всей партии добавляют ровно 5 %. На катушке осталось M граммов. Требуемый расход равен $R = Nm \times 1,05$. Если M не меньше R , материала хватает; вывести остаток. Иначе определить недостающую массу. Равенство M и R означает, что материала хватает без остатка. Для массы и расчёта расхода использовать decimal, чтобы десятичные значения сравнивались без двоичного округления.

Ввести: M – остаток на катушке, г; m – расход на одну деталь, г; N – число деталей.

Вывести: требуемую массу с двумя знаками; заключение; остаток либо недостачу в граммах.

Контрольный пример. Вход: $M = 500$, $m = 40$, $N = 12$. Результат: Требуется 504,00 г; материала не хватает; недостача 4,00 г.

Практическая работа 4

После печати партии получены значения фактического расхода для отдельных моделей. Цена материала P дана за килограмм. В цикле найти суммарный расход и определить модель, на которую ушло больше всего материала. При одинаковом максимальном расходе вывести первую по порядку модель. Стоимость материала всей партии равна сумме масс в граммах, делённой на 1000 и умноженной на P . Повторно добавлять запас 5 % к фактическому расходу не нужно.

Ввести: N – число моделей; P – цена, руб/кг; затем масса материала для каждой модели, г.

Вывести: общую массу; стоимость с двумя знаками; номер модели с максимальным расходом и её расход.

Контрольный пример. Вход: $N = 4$, $P = 1800$; массы 20; 35,5; 18; 26,5 г. Результат: Всего 100,00 г; стоимость 180,00 руб.; модель № 2, расход 35,50 г.

Вариант 8 Размещение деталей на платформе

Практическая работа 3

На платформе X на Y миллиметров размещают квадратные детали со стороной a . От края оставляют по 5 мм, между соседними деталями – 5 мм. Если $X - 10$ или $Y - 10$ меньше a , деталь не помещается. Иначе nX равно целой части $(X - 5) / (a + 5)$, nY – целой части $(Y - 5) / (a + 5)$. Общее количество $K = nX \cdot nY$. Занятая площадь: $100Ka^2 / (XY)$ процентов. Зазоры в занятую площадь не включать.

Ввести: X и Y – размеры платформы, мм; a – сторона детали, мм. Все значения целые.

Вывести: число деталей по X и Y ; общее количество; занятую площадь в процентах с двумя знаками либо сообщение «Не помещается».

Контрольный пример. Вход: $X = 220$, $Y = 220$, $a = 50$. Результат: По $X - 3$, по $Y - 3$, всего 9; занято 46,49 %.

Практическая работа 4

За один запуск принтер изготавливает не больше K деталей. Заказы выполняются отдельно: свободные места последнего запуска нельзя занять деталями другого заказа. Вводить количество деталей очередного заказа до нуля, но не больше двадцати заказов. Для каждого заказа найти число запусков с округлением вверх. В цикле накопить число деталей, запусков и незанятых мест. Ноль завершает ввод и не создаёт заказ. При отсутствии заказов все итоги равны нулю.

Ввести: K – вместимость платформы, шт.; затем размеры заказов, в конце 0.

Вывести: общее число деталей, запусков и свободных мест во всех запусках.

Контрольный пример. Вход: $K = 9$; заказы 18, 10, 5, 0. Результат: Деталей 33; запусков 5; свободных мест 12.

Вариант 9 Время и энергия печати

Практическая работа 3

Для условной оценки времени заданы три профиля печати. Профиль 1 имеет производительность 2 см³/мин, профиль 2 – 1,5 см³/мин, профиль 3 – 1 см³/мин. Производительность выбрать оператором switch. Если объём материала V, время подготовки T_п, а выбранная производительность q, полное время равно $T = T_{п} + V / q$. Подготовку учитывать один раз. Эти значения описывают учебную модель; расчёт не учитывает перемещения без подачи материала и изменение скорости на отдельных участках.

Ввести: номер профиля; V – объём, см³; T_п – время подготовки, мин.

Вывести: производительность выбранного профиля и полное время с двумя знаками.

Контрольный пример. Вход: Профиль 2, V = 30, T_п = 6. Результат: Производительность 1,5 см³/мин; полное время 26,00 мин.

Практическая работа 4

Принтер последовательно выполнил N заданий. Для каждого известно полное время работы в минутах. Считать, что во время всех заданий принтер потребляет постоянную мощность P ватт. В цикле накопить время и найти самое длительное задание. При равенстве времён сохранить номер первого. Энергию вычислить после цикла: $E = (P / 1000) \times (T / 60)$, где T – сумма минут. Между заданиями принтер выключен; дополнительный расход энергии не учитывается.

Ввести: N – число заданий; P – мощность, Вт; затем время каждого задания, мин.

Вывести: суммарное время; энергию в кВт·ч с тремя знаками; номер и продолжительность самого долгого задания.

Контрольный пример. Вход: N = 3, P = 250; времена 60, 90, 30 мин. Результат: Всего 180 мин; энергия 0,750 кВт·ч; самое долгое задание № 2, 90 мин.

Вариант 10 Материал печатной модели

Практическая работа 3

Материал выбирается по коду: 1 – PLA, плотность 1,24 г/см³ и цена 1800 руб/кг; 2 – ABS, 1,04 г/см³ и 2000 руб/кг; 3 – PETG, 1,27 г/см³ и 2200 руб/кг. Значения заданы для этой задачи. Оператором switch выбрать название, плотность и цену. Масса одной модели равна объёму V, умноженному на плотность. По количеству N определить массу партии и стоимость материала. Для стоимости перевести граммы в килограммы; потери материала не добавлять.

Ввести: код материала; V – объём одной модели, см³; N – количество моделей.

Вывести: название материала; массу одной модели и партии в граммах; стоимость партии с двумя знаками.

Контрольный пример. Вход: Код 3, V = 40, N = 5. Результат: PETG; одна модель 50,80 г; партия 254,00 г; стоимость 558,80 руб.

Практическая работа 4

В журнале печати указаны код материала и фактический расход для каждой работы. Названия и цены соответствуют предыдущей задаче. Обработать записи циклом и отдельно накопить массы PLA, ABS и PETG. По трём итоговым массам вычислить общую стоимость. Если материал не использовался, его масса должна остаться нулевой. Одна запись относится только к одному материалу. Расход задан в граммах; цену применять после перевода массы в килограммы.

Ввести: N – число работ; для каждой работы код материала и расход, г.

Вывести: расход каждого из трёх материалов; общий расход и стоимость с двумя знаками.

Контрольный пример. Вход: N = 4; пары код/масса: 1/100, 2/50, 1/25, 3/80. Результат: PLA – 125 г, ABS – 50 г, PETG – 80 г; всего 255 г; стоимость 501,00 руб.

Вариант 11 Отклонение измеренного размера

Практическая работа 3

Для детали задан номинальный размер D . Измеренное отклонение x и допустимое отклонение a заданы целыми микрометрами. Поле допуска симметрично: от $-a$ до $+a$ включительно. Если x меньше $-a$, размер ниже допуска; если больше a – выше допуска; иначе размер в допуске. Фактический размер равен $D + x / 1000$, где деление выполняется с сохранением дробной части. Для детали вне допуска определить, на сколько микрометров нарушена ближайшая граница.

Ввести: D – номинал, мм; x – отклонение, мкм; a – допустимое отклонение, мкм.

Вывести: фактический размер с тремя знаками; заключение; при выходе за допуск – превышение границы.

Контрольный пример. Вход: $D = 30$, $x = -18$, $a = 15$. Результат: Размер 29,982 мм; ниже допуска на 3 мкм.

Практическая работа 4

Измерены отклонения N деталей с таким же симметричным полем допуска. В цикле найти среднее отклонение со знаком, наибольшую абсолютную величину отклонения и число деталей вне допуска. При вычислении среднего отрицательные значения не заменять положительными. Модуль нужен только для поиска наибольшей величины и проверки границ. Отклонения ровно $-a$ и $+a$ входят в допуск. Среднее выводится после обработки всей серии.

Ввести: N ; a – граница допуска, мкм; затем N целых отклонений, мкм.

Вывести: среднее отклонение с двумя знаками; максимальный модуль; число деталей вне допуска.

Контрольный пример. Вход: $N = 5$, $a = 15$; отклонения $-18, -5, 0, 12, 20$. Результат: Среднее +1,80 мкм; максимальный модуль 20 мкм; вне допуска 2 детали.

Вариант 12 Предельные размеры детали

Практическая работа 3

По номинальному размеру D и нижнему и верхнему отклонениям e_i, e_s определить предельные размеры: $D_{\min} = D + e_i / 1000$, $D_{\max} = D + e_s / 1000$. Отклонения заданы целыми микрометрами и могут быть отрицательными. Допуск равен $e_s - e_i$. Определить положение поля относительно номинала: целиком выше при $e_i > 0$, целиком ниже при $e_s < 0$, в остальных случаях номинал входит в поле. Равенство одной из границ нулю относится к последнему случаю.

Ввести: D – номинальный размер, мм; e_i и e_s – отклонения, мкм, при $e_i \leq e_s$.

Вывести: оба предельных размера с тремя знаками; допуск в микрометрах; положение поля относительно номинала.

Контрольный пример. Вход: $D = 45$, $e_i = -14$, $e_s = 25$. Результат: $D_{\min} = 44,986$ мм; $D_{\max} = 45,025$ мм; допуск 39 мкм; номинал входит в поле.

Практическая работа 4

В таблице размеров заданы N пар отклонений e_i и e_s . Для каждой пары вычислить ширину поля допуска $e_s - e_i$. В цикле определить среднюю, наименьшую и наибольшую ширину, а также номера соответствующих размеров. При одинаковом минимуме или максимуме сохранять первый встретившийся номер. Нумерация начинается с единицы. Для среднего использовать вещественное деление, даже если все ширины целые.

Ввести: N ; затем для каждого размера нижнее и верхнее отклонения e_i, e_s в микрометрах при $e_i \leq e_s$.

Вывести: средний допуск с двумя знаками; минимальный и максимальный допуски и номера размеров.

Контрольный пример. Вход: $N = 3$; пары e_i/e_s : $-10/20, -5/15, -20/30$. Результат: Средний допуск 33,33 мкм; минимум 20 мкм у № 2; максимум 50 мкм у № 3.

Вариант 13 Размерные группы деталей

Практическая работа 3

Поле отклонений от L до U разделено на две одинаковые размерные группы. Граница между ними равна $c = (L + U) / 2$ и может быть дробной. Если измеренное отклонение x находится за пределами $[L; U]$, деталь не входит ни в одну группу. Иначе первая группа содержит значения от L включительно до c не включительно, вторая – от c до U включительно. Сначала проверить выход за общее поле, затем определить группу.

Ввести: L и U – границы в мкм, $L < U$; x – измеренное отклонение, мкм. Все три значения целые.

Вывести: границу c с одним знаком; номер группы либо сообщение о выходе за поле.

Контрольный пример. Вход: $L = -20$, $U = 20$, $x = 0$. Результат: Граница 0,0 мкм; группа 2.

Практическая работа 4

Для учебного комплекта нужна одна деталь из первой группы и одна из второй. Здесь границы фиксированы: первая группа от -20 мкм включительно до 0 не включительно; вторая от 0 до $+20$ включительно. Обработать N отклонений циклом, отдельно подсчитать обе группы и детали вне поля. Число полных комплектов k равно меньшему из двух групповых счётчиков. Подходящий остаток равен сумме этих счётчиков минус $2k$. Детали вне поля в остаток не включать.

Ввести: N ; затем N целых отклонений, мкм.

Вывести: численность обеих групп; число деталей вне поля; полные комплекты и подходящий остаток.

Контрольный пример. Вход: $N = 6$; отклонения $-20, -5, 0, 12, 20, 25$. Результат: Группы: 2 и 3; вне поля 1; комплектов 2; подходящий остаток 1.

Вариант 14 Зазор в измеренной паре

Практическая работа 3

Отверстие и вал имеют одинаковый номинальный размер. Их измеренные отклонения x и y заданы целыми микрометрами, поэтому фактическая разность размеров равна $S = x - y$. При $S < 0$ получается натяг величиной $-S$; при $S = 0$ размеры совпадают. Положительный S означает зазор. Сравнить его с заданным пределом $S_{\max}$: равенство разрешено, превышение нужно указать численно. Для натяга предел положительного зазора не применяется.

Ввести: x – отклонение отверстия, мкм; y – отклонение вала, мкм; $S_{\max}$ – допустимый зазор, мкм.

Вывести: разность S со знаком; вид соединения; для слишком большого зазора – превышение предела.

Контрольный пример. Вход: $x = 20$, $y = 5$, $S_{\max} = 10$. Результат: $S = +15$ мкм; зазор; превышение предела 5 мкм.

Практическая работа 4

Проверяются N заранее подобранных пар отверстие–вал. Для каждой считать S по разности отклонений. В этой серии подходящими считаются пары с S от 0 до 20 мкм включительно. Остальные разделить на пары с натягом и пары с зазором больше 20 мкм. В цикле накопить три счётчика и сумму S . Затем вычислить среднюю разность со знаком. Отрицательные значения входят в сумму без изменения знака.

Ввести: N ; для каждой пары целые отклонения отверстия и вала в микрометрах.

Вывести: количество подходящих пар, пар с натягом и слишком большим зазором; среднюю разность с двумя знаками.

Контрольный пример. Вход: $N = 4$; пары x/y : 20/5, 0/10, 10/10, 30/0. Результат: Подходящих 2; с натягом 1; с большим зазором 1; среднее $+8,75$ мкм.

Вариант 15 Контроль массы деталей

Практическая работа 3

Для детали заданы номинальная масса m_0 и допустимое отклонение p процентов в обе стороны. Предельные массы равны $m_0(1 - p / 100)$ и $m_0(1 + p / 100)$. Сравнить измеренную массу m с этими границами: ниже нижней, в пределах или выше верхней. Обе границы входят в допустимый интервал. Найти также относительное отклонение $100(m - m_0) / m_0$ со знаком. Для исходных масс и расчётов использовать decimal; p задан в пределах от 0 до 100.

Ввести: m_0 – номинальная масса, г; m – измеренная масса, г; p – допустимое отклонение, %.

Вывести: предельные массы с двумя знаками; относительное отклонение с двумя знаками; заключение.

Контрольный пример. Вход: $m_0 = 200$, $m = 188$, $p = 5$. Результат: Границы 190,00–210,00 г; отклонение –6,00 %; масса ниже нижней границы.

Практическая работа 4

Взвешены N одинаковых деталей. Номинальная масса m_0 одинакова для всей серии. В цикле вычислить общую массу, подсчитать детали легче номинала и найти наибольшее абсолютное отклонение $|m - m_0|$ в граммах. После ввода определить среднюю массу. Деталь с массой ровно m_0 не относится к числу более лёгких. Общую массу накапливать из исходных значений, не округляя каждое измерение отдельно.

Ввести: N ; m_0 – номинальная масса, г; затем массы N деталей.

Вывести: общую и среднюю массы; число деталей легче номинала; максимальное отклонение в граммах. Дробные результаты – с двумя знаками.

Контрольный пример. Вход: $N = 5$, $m_0 = 100$; массы 99; 101; 100; 98,5; 101,5. Результат: Общая масса 500,00 г; средняя 100,00 г; легче номинала 2; максимальное отклонение 1,50 г.

Вариант 16 Короба для готовой продукции

Практическая работа 3

В короб помещается s деталей. Для заказа из q деталей сначала заполняют полные короба, затем при необходимости используют один неполный. Определите число коробов и свободных мест. Если остатка нет, выведите «Все короба полные». Иначе сравните заполнение последнего короба с половиной вместимости: меньше половины либо не меньше половины. При точном делении свободных мест нет.

Ввести: q – количество деталей; s – вместимость короба, шт. Обе величины целые положительные.

Вывести: Количество полных коробов, общее число коробов, свободные места в последнем коробе и заключение о заполнении.

Контрольный пример. Вход: $q = 53$; $s = 12$. Результат: Полных коробов 4; всего 5; свободных мест 7; последний короб заполнен меньше чем наполовину.

Практическая работа 4

Для каждого короба известны длина a и ширина b крышки. Упаковочную ленту накладывают по прямоугольному контуру, добавляя 20 см на соединение концов. Расход равен $2(a + b) + 20$ см. Циклом for обработайте N пар размеров. Найдите общий расход и наиболее затратный короб. При одинаковом максимуме сохраняйте первый номер. Ширина ленты в расчёт не входит.

Ввести: N от 1 до 20; затем N пар целых положительных размеров a и b в сантиметрах.

Вывести: Общий расход в метрах, средний расход на короб в сантиметрах, номер и расход наиболее затратного короба.

Контрольный пример. Вход: $N = 4$; пары a/b : 40/30, 50/20, 25/25, 60/40. Результат: Всего 6,60 м; среднее 165,00 см; максимум у короба № 4, расход 220 см.

Вариант 17 Размещение и движение складского запаса

Практическая работа 3

На полке осталось R миллиметров свободной длины. Изделия длиной l устанавливают в один ряд без промежутков. Нужно разместить q изделий. Найдите вместимость свободного участка целочисленным делением. Если места достаточно, разместите весь заказ; если помещается только часть, разместите эту часть; при нулевой вместимости не размещайте ничего. Длину остатка считайте после фактического размещения.

Ввести: R – свободная длина, мм; l – длина изделия, мм; q – количество изделий. Значения целые положительные.

Вывести: Размещено и осталось изделий, свободная длина полки и заключение: полностью, частично или места нет.

Контрольный пример. Вход: $R = 250$; $l = 60$; $q = 6$. Результат: Размещено 4; осталось 2; свободно 10 мм; заказ размещён частично.

Практическая работа 4

Обработайте журнал из N складских операций. Положительное число означает поступление, отрицательное – выдачу. Выдачу, превышающую текущий остаток, отклоните целиком и переходите к следующей записи. Остальные операции изменяют запас. Циклом `for` подсчитайте принятые поступления и выдачи отдельно. Отклонённая выдача не входит в сумму выданного материала.

Ввести: S – начальный запас, шт.; N от 1 до 20; затем N целых изменений запаса.

Вывести: Конечный запас, всего принято и выдано, число отклонённых операций. Нулевая запись запас не меняет.

Контрольный пример. Вход: $S = 30$; $N = 5$; операции: $-12, 8, -40, -10, 5$. Результат: Запас 21 шт.; принято 13; выдано 22; отклонена 1 операция.

Вариант 18 Поставка и накопление партии

Практическая работа 3

Для выполнения заказа нужно q деталей, на складе имеется s . Недостающее количество заказывают одной партией размером 50, 100 или 200 штук. Выберите наименьшую подходящую партию цепочкой `if / else if`. Если запаса хватает, закупка не нужна. Недостающее количество по условию не превышает 200. После поставки весь заказ сразу передают потребителю.

Ввести: q – потребность, шт.; s – имеющийся запас, шт. q положительное, s неотрицательное целое.

Вывести: Недостающее количество, размер закупки и остаток на складе после выполнения заказа.

Контрольный пример. Вход: $q = 102$; $s = 35$. Результат: Не хватает 67 шт.; закупить 100 шт.; остаток после выдачи 33 шт.

Практическая работа 4

Для отгрузки накапливают не менее B деталей из поступающих партий. Размер очередной партии вводится только тогда, когда требуемое количество ещё не набрано. Используйте `while` и принимайте каждую партию целиком. Завершите ввод при достижении B либо после двадцатой партии. Последняя принятая партия может создать превышение; делить её нельзя.

Ввести: B – требуемое количество; затем положительные целые размеры поступающих партий, не более 20 значений.

Вывести: Число принятых партий, накопленное количество. Дополнительно выведите превышение или недостающее количество, если лимит ввода исчерпан.

Контрольный пример. Вход: $B = 90$; поступления: 25, 30, 40. Результат: Приняты 3 партии; накоплено 95 шт.; превышение 5 шт.; дальнейший ввод не требуется.

Вариант 19 Перевозка готовой продукции

Практическая работа 3

Выберите транспорт по коду: 1 – грузоподъёмность 1,5 т и стоимость рейса 1200 руб.; 2 – 5 т и 2000 руб.; 3 – 10 т и 3500 руб. Параметры задайте через switch. Весь груз перевозят выбранным транспортом за несколько рейсов. Число рейсов округлите вверх; каждый рейс оплачивается полностью независимо от загрузки.

Ввести: Код транспорта и общая масса груза m , т. Масса положительная, допускается дробная часть.

Вывести: Число рейсов, общая стоимость и среднее использование грузоподъёмности: $m / (\text{рейсы} \times \text{грузоподъёмность}) \times 100 \%$.

Контрольный пример. Вход: Код = 2; $m = 12$ т. Результат: Рейсов 3; стоимость 6000,00 руб.; использование грузоподъёмности 80,00 %.

Практическая работа 4

Маршрут состоит из N участков. Автомобиль проходит каждый участок со средней скоростью 40 км/ч, после каждого участка выполняет разгрузку продолжительностью 5 минут, включая последний. Циклом for считайте расстояния, суммируйте длину маршрута и время. Определите самый длинный участок. При одинаковой длине нескольких участков выбирайте первый по порядку ввода.

Ввести: N от 1 до 20; затем длины N участков в километрах. Нулевая длина допускается.

Вывести: Общая длина маршрута, общее время с разгрузками в минутах, номер и длина самого длинного участка.

Контрольный пример. Вход: $N = 4$; участки: 20, 0, 40, 10 км. Результат: Маршрут 70,00 км; время 125,00 мин; самый длинный участок № 3, 40,00 км.

Вариант 20 Масса автомобиля и загрузка кузова

Практическая работа 3

Для въезда на площадку установлена предельная полная масса P . Полная масса автомобиля складывается из собственной массы a и массы груза b . Рассчитайте её и сравните с пределом. При меньшей массе сообщите запас, при равенстве – отсутствие запаса, при большей – величину превышения. Во всех случаях выводите фактическую полную массу.

Ввести: a – масса автомобиля без груза; b – масса груза; P – предел. Все значения целые, в килограммах.

Вывести: Полная масса, заключение по сравнению с P и величина запаса либо превышения в килограммах.

Контрольный пример. Вход: $a = 1600$; $b = 2400$; $P = 4500$ кг. Результат: Полная масса 4000 кг; предел не превышен; запас 500 кг.

Практическая работа 4

Грузят ящики в заданном порядке. Перед добавлением очередного ящика проверьте, не превысит ли суммарная масса грузоподъёмность Q . Если ящик не помещается, прекратите ввод через break: пропускать его и искать более лёгкий нельзя. Точное заполнение разрешено. Используйте while; допускается не более N предложенных ящиков. Непоместившийся ящик загруженным не считается.

Ввести: Q – грузоподъёмность, кг; N от 1 до 20; далее положительные целые массы ящиков.

Вывести: Количество загруженных ящиков, их масса и свободная грузоподъёмность. При остановке укажите номер непоместившегося ящика.

Контрольный пример. Вход: $Q = 1000$; $N = 5$; введены массы 250, 400, 500 кг. Результат: Загружены 2 ящика массой 650 кг; свободно 350 кг; остановка на ящике № 3.

Вариант 21 Потребление электроэнергии

Практическая работа 3

По коду оборудования выберите постоянную мощность: 1 – фрезерный станок 5,5 кВт; 2 – сверлильный станок 2,2 кВт; 3 – шлифовальный станок 4,0 кВт. Используйте switch. За время работы t потреблённая энергия равна $E = P \times t$. Стоимость определяется умножением энергии на тариф. Периодов простоя в этой задаче нет.

Ввести: Код оборудования; t – время работы, ч; r – тариф, руб. за кВт·ч. t и r положительные.

Вывести: Мощность выбранного оборудования, потреблённая энергия в кВт·ч и стоимость работы. Вещественные результаты выводите с двумя знаками.

Контрольный пример. Вход: Код = 1; $t = 3,5$ ч; $r = 6$ руб. за кВт·ч. Результат: Мощность 5,50 кВт; энергия 19,25 кВт·ч; стоимость 115,50 руб.

Практическая работа 4

Счётчик записывали несколько раз через одинаковые интервалы. Показания возрастают либо остаются прежними. Первый ввод задаёт начальное показание. Циклом for обрабатывайте остальные и находите расход за каждый интервал как разность текущего и предыдущего показаний. Показания храните в decimal. Хранить все показания не требуется. Найдите общий и средний расход, а также интервал максимального расхода.

Ввести: N – число показаний от 2 до 20; затем N показаний счётчика в кВт·ч.

Вывести: Общий расход, средний расход за интервал, максимальный расход и номер первого интервала с этим значением.

Контрольный пример. Вход: $N = 5$; показания: 1200, 1212, 1220, 1235, 1240. Результат: Общий расход 40,00; средний 10,00; максимальный 15,00 кВт·ч, интервал № 3.

Вариант 22 Время проходов и ресурс инструмента

Практическая работа 3

Выберите режим обработки через switch: код 1 – один проход с подачей 120 мм/мин; код 2 – два прохода с подачей 90 мм/мин; код 3 – три прохода с подачей 60 мм/мин. На каждом проходе путь инструмента равен длине детали L плюс 5 мм. Время детали равно пути, умноженному на число проходов и делённому на подачу.

Ввести: Код режима; L – длина детали, мм; q – количество одинаковых деталей. L положительная, q целое положительное.

Вывести: Время обработки одной детали и всей партии в минутах с двумя знаками. Возврат инструмента не учитывается.

Контрольный пример. Вход: Код = 2; $L = 180$ мм; $q = 12$. Результат: Одна деталь 4,11 мин; вся партия 49,33 мин.

Практическая работа 4

Инструмент имеет ресурс T минут. Последовательно выполните N операций известной продолжительности. Если оставшегося ресурса недостаточно для следующей операции, замените инструмент перед её началом. Операция не делится между инструментами; при точном равенстве замена перед ней не нужна. Циклом for подсчитайте расход инструмента и время. Каждый отдельный ввод не превышает T . Для времени используйте decimal.

Ввести: T – ресурс нового инструмента, мин; N от 1 до 20; затем N положительных продолжительностей операций.

Вывести: Всего использовано инструментов, выполнено замен, суммарное время и оставшийся ресурс последнего инструмента.

Контрольный пример. Вход: $T = 60$; $N = 5$; операции: 20, 30, 15, 40, 10 мин. Результат: Использовано 3 инструмента; замен 2; время 115,00 мин; оставшийся ресурс 50,00 мин.

Вариант 23 Фонд времени рабочей смены

Практическая работа 3

Смена длится 480 минут. Из неё исключается перерыв b . На одну деталь требуется t минут, нужно изготовить q деталей. Рассчитайте доступное время и продолжительность задания. Сравните их: работа укладывается с запасом, занимает всё доступное время либо требует дополнительного времени. Равенство относится к полному использованию смены. Число деталей не округляйте и не уменьшайте. Для времени используйте decimal.

Ввести: b – перерыв, от 0 до 479 мин; t – время детали, мин; q – положительное целое количество.

Вывести: Доступное время, время задания, заключение и величина свободного либо дополнительного времени в минутах.

Контрольный пример. Вход: $b = 30$; $t = 7$; $q = 60$. Результат: Доступно 450,00 мин; задание 420,00 мин; укладывается в смену; запас 30,00 мин.

Практическая работа 4

За смену зарегистрировано N остановок оборудования. Циклом for суммируйте их продолжительности и определите самую долгую остановку. Отдельно подсчитайте остановки длительностью не менее 10 минут. Суммарный простой по условию не превышает 480 минут. Рабочее время равно 480 минут минус простой, его доля в смене вычисляется относительно 480 минут.

Ввести: N от 1 до 20; затем N неотрицательных продолжительностей остановок, мин. Нулевую запись также обработайте.

Вывести: Общий простой, число остановок не короче 10 минут, максимальный простой, рабочее время и его доля в процентах.

Контрольный пример. Вход: $N = 5$; остановки: 5, 12, 0, 18, 5 мин. Результат: Простой 40,00 мин; длительных остановок 2; максимум 18,00 мин; работа 440,00 мин; доля 91,67 %.

Вариант 24 Распределение заданий между станками

Практическая работа 3

Задание из q деталей можно направить на станок А или Б. На А подготовка длится 10 минут, обработка детали – 2 минуты. На Б подготовка длится 25 минут, обработка детали – 1 минуту. Станки освобождаются через a и b минут соответственно. Сравните время окончания нового задания, учитывая ожидание. При равенстве разрешён любой станок.

Ввести: a и b – ожидание освобождения станков, мин; q – количество деталей. Все значения целые; a и b неотрицательные, q положительное.

Вывести: Через сколько минут задание закончится на каждом станке и какой станок обеспечит более раннее окончание.

Контрольный пример. Вход: $a = 35$; $b = 10$; $q = 20$. Результат: Станок А: 85 мин; станок Б: 55 мин; выбрать Б, окончание через 55 мин.

Практическая работа 4

Два одинаковых станка первоначально свободны. Задания поступают в заданном порядке. Каждое целиком назначается станку с меньшей суммой уже назначенных минут; при равенстве выбирайте А. Циклом for считайте продолжительности и сразу увеличивайте загрузку выбранного станка. Переносить прежние задания после назначения нельзя. Время завершения равно большей загрузке.

Ввести: N от 1 до 20; затем N положительных целых продолжительностей заданий в минутах.

Вывести: Для каждого задания выбранный станок; затем количество заданий и загрузку каждого станка, время завершения всех работ.

Контрольный пример. Вход: $N = 5$; задания: 20, 10, 15, 30, 5 мин. Результат: Назначения: А, Б, Б, А, Б. На А 2 задания и 50 мин; на Б 3 и 30 мин; окончание 50 мин.

Вариант 25 Сроки выполнения заказов

Практическая работа 3

До срока готовности заказа осталось D минут. Нужно изготовить q одинаковых деталей, каждая занимает t минут. Подготовительных операций нет. Рассчитайте время заказа и резерв $D - q \times t$. Если резерв отрицательный, заказ опаздывает; от 0 до 30 минут включительно – малый резерв; больше 30 минут – достаточный резерв. Ровно нулевой резерв означает выполнение точно к сроку. Для времени используйте decimal.

Ввести: D – время до срока, мин; q – целое положительное количество; t – время одной детали, мин.

Вывести: Время и резерв со знаком с двумя знаками; заключение.

Контрольный пример. Вход: $D = 150$; $q = 30$; $t = 4$. Результат: Время 120,00 мин; резерв 30,00 мин; малый резерв.

Практическая работа 4

На одном рабочем месте последовательно выполняют N заказов без перерывов. Для всех установлен общий срок D минут от начала работы. Циклом for суммируйте продолжительности: накопленная сумма является временем окончания очередного заказа. Заказ закончен вовремя, если эта сумма не больше D . После первого опоздания обработайте остальные записи. Сохраните номер первого опоздавшего заказа. Для времени используйте decimal.

Ввести: N от 1 до 20; D – общий срок, мин; затем N положительных продолжительностей заказов.

Вывести: Число заказов вовремя и с опозданием, время завершения всех заказов и номер первого опоздавшего; при отсутствии опозданий – 0.

Контрольный пример. Вход: $N = 5$; $D = 60$; продолжительности: 15, 20, 30, 10, 5 мин. Результат: Вовремя 2; с опозданием 3; общее время 80,00 мин; первый опоздавший заказ № 3.

Вариант 26 Выполнение плана и почасовой выпуск

Практическая работа 3

План участка равен p годным деталям. Изготовлено q деталей, из них r признаны негодными. Определите годный выпуск $q - r$ и процент выполнения плана. При годном выпуске меньше p сообщите недостающее количество, при равенстве – точное выполнение, при большем выпуске – перевыполнение. В проценте учитываются только годные детали, а не весь изготовленный объём.

Ввести: p – план; q – изготовлено; r – негодные. Все значения целые, p положительное, $0 \leq r \leq q$.

Вывести: Годный выпуск, процент выполнения с двумя знаками, заключение и недостающее количество либо перевыполнение.

Контрольный пример. Вход: $p = 120$; $q = 135$; $r = 9$. Результат: Годных 126 шт.; выполнение 105,00 %; перевыполнение 6 шт.

Практическая работа 4

Даны результаты выпуска за N последовательных часов. Найдите наиболее длинный непрерывный отрезок часов, на котором каждый следующий результат строго больше предыдущего. Равенство разрывает рост. Один час считается отрезком длины один. Используйте for, сохраняя только предыдущее значение, длину текущего роста и лучший результат. При одинаковой длине оставьте первый найденный отрезок.

Ввести: N от 1 до 20; затем N неотрицательных целых значений почасового выпуска. Часы нумеруются с 1.

Вывести: Суммарный выпуск; длину самого длинного роста и номера его первого и последнего часов.

Контрольный пример. Вход: $N = 7$; выпуск: 5, 7, 9, 4, 6, 8, 10. Результат: Всего 49 шт.; самый длинный рост 4 часа, с часа № 4 по час № 7.

Вариант 27 Пополнение материального запаса

Практическая работа 3

Ежедневно расходуют d единиц материала, поставка занимает L дней. Минимальный запас на срок поставки равен $d \times L$. Сравните имеющийся запас s с этим значением. При недостатке определите, сколько единиц не хватает, при равенстве выведите «Запас точно на срок поставки», иначе укажите превышение минимального запаса. Дополнительно найдите число полных дней обеспеченности целочисленным делением.

Ввести: s – запас; d – суточный расход; L – срок поставки. Значения целые, d и L положительные, s неотрицательное.

Вывести: Минимальный запас, полные дни обеспеченности и заключение с величиной недостатка либо превышения.

Контрольный пример. Вход: $s = 83$; $d = 12$; $L = 7$. Результат: Минимум 84 единицы; обеспеченность 6 полных дней; не хватает 1 единицы.

Практическая работа 4

Начальный запас составляет 50 единиц. После каждого дневного расхода проверяйте остаток. Если он стал меньше 20, немедленно добавьте партию из 50 единиц; при остатке ровно 20 пополнения нет. Циклом `for` обработайте N дней. Один дневной расход не превышает 20 единиц, поэтому запас не становится отрицательным. Пополнение возможно и после последнего дня.

Ввести: N от 1 до 20; затем N целых дневных расходов от 0 до 20 единиц.

Вывести: Общий расход, количество пополнений, всего получено материала и конечный запас после всех пополнений.

Контрольный пример. Вход: $N = 6$; расходы: 15, 20, 10, 20, 5, 10. Результат: Израсходовано 80; пополнений 1; получено 50; конечный запас 20 единиц.

Вариант 28 Масса изделий и результаты взвешивания

Практическая работа 3

Груз массой m расположен между двумя опорами на расстоянии L друг от друга. Его центр тяжести находится на расстоянии x от левой опоры. В учебном расчёте масса распределяется по формулам: $m_L = m(L - x) / L$, $m_P = mx / L$. Определите, на какую опору приходится большая масса, либо сообщите о равенстве. Вычисления выполняйте в `decimal`.

Ввести: m – масса, кг; L – расстояние между опорами, м; x – положение центра тяжести, м, от 0 до L включительно.

Вывести: Масса на каждой опоре, их разность по модулю и заключение. Массы выводите с двумя знаками.

Контрольный пример. Вход: $m = 240$ кг; $L = 2$ м; $x = 0,75$ м. Результат: Слева 150,00 кг; справа 90,00 кг; больше слева; разность 60,00 кг.

Практическая работа 4

В серии взвешиваний вычислите обычную среднюю массу и среднюю после исключения одного наименьшего и одного наибольшего результата. Используйте `for`, сумму и два экстремума; массив не нужен. Если одинаковый минимум или максимум встречается несколько раз, исключается только одно значение каждого вида. При одинаковых массах всех изделий всё равно исключаются ровно два взвешивания.

Ввести: N от 3 до 20; затем N положительных целых масс изделий в граммах.

Вывести: Минимальная и максимальная массы, обычное среднее и среднее без двух крайних значений; средние – с двумя знаками.

Контрольный пример. Вход: $N = 5$; массы: 1010, 980, 1050, 1000, 960 г. Результат: Минимум 960 г; максимум 1050 г; среднее 1000,00 г; после исключения 996,67 г.

Вариант 29 Трудоёмкость и выполнение нормы

Практическая работа 3

Бригаде из n работников поручена работа трудоёмкостью W нормо-часов. Каждый располагает H часами рабочего времени. Фонд бригады равен $n \times H$. Рассчитайте загрузку $W / (n \times H) \times 100 \%$. При загрузке меньше 70 % бригада недогружена, от 70 до 100 % включительно – загрузка допустимая, выше 100 % – перегрузка. Пограничные значения не округляйте перед сравнением. Вычисления выполняйте в decimal.

Ввести: W – трудоёмкость, нормо-ч; n – целое число работников; H – время одного работника, ч. Значения положительные.

Вывести: Фонд бригады, загрузка и заключение; дополнительно разность фонда и трудоёмкости со знаком.

Контрольный пример. Вход: $W = 28$; $n = 4$; $H = 8$. Результат: Фонд 32,00 ч; загрузка 87,50 %; допустимая; свободный фонд +4,00 ч.

Практическая работа 4

За смену каждый работник располагает 480 минутами. Норма времени составляет 12 минут на деталь. По выпуску каждого работника вычислите выполнение нормы: количество $\times 12 / 480 \times 100 \%$. Циклом for обработайте N работников, сразу выводя их проценты. Подсчитайте выполнивших норму не менее чем на 100 % и определите лучший результат. При равенстве выберите первого работника.

Ввести: N от 1 до 20; затем N неотрицательных целых количеств изготовленных деталей.

Вывести: Процент каждого работника, общий выпуск, число выполнивших норму, номер лучшего работника и его процент.

Контрольный пример. Вход: $N = 4$; выпуск: 35, 40, 50, 30 шт. Результат: Проценты: 87,50; 100,00; 125,00; 75,00. Всего 155 шт.; норму выполнили 2; лучший № 3, 125,00 %.

Вариант 30 Стоимость заказов и дни отгрузки

Практическая работа 3

Стоимость заказа до скидки равна количеству q , умноженному на цену p . Для количества от 50 до 99 включительно скидка равна 5 %, от 100 – 10 %, иначе скидки нет. За срочность начисляется 15 % от стоимости уже после скидки. Код 1 означает обычный заказ, код 2 – срочный. Сначала выберите скидку ветвлением, затем учтите срочность.

Ввести: q – положительное целое количество; p – цена единицы, руб.; код срочности 1 или 2.

Вывести: Сумма скидки, доплата за срочность и окончательная стоимость с двумя знаками. Промежуточные суммы не округляйте.

Контрольный пример. Вход: $q = 80$; $p = 250$ руб.; код = 2. Результат: Скидка 1000,00 руб.; доплата 2850,00 руб.; стоимость 21850,00 руб.

Практическая работа 4

Заказы перечислены по дням отгрузки в неубывающем порядке. Все заказы одного дня отправляют одной общей отгрузкой. Циклом for посчитайте количество таких отгрузок и найдите день с наибольшим числом заказов. Храните текущий день и длину группы одинаковых дат. При одинаковом максимуме выбирайте более ранний день. Менять порядок введённых значений или применять сортировку не требуется.

Ввести: N от 1 до 20; затем N номеров дней от 1 до 31, упорядоченных по возрастанию с возможными повторениями.

Вывести: Число общих отгрузок, день с наибольшим количеством заказов и число заказов в этот день.

Контрольный пример. Вход: $N = 8$; дни: 3, 3, 5, 5, 5, 8, 8, 12. Результат: Отгрузок 4; наиболее загружен день 5, на него приходится 3 заказа.

Оформление отчёта

К каждой работе приложите Program.cs и краткий отчёт. В отчёте укажите номер варианта, условие своей задачи, используемые формулы и результаты проверок. Достаточно описать порядок вычислений обычным текстом; блок-схема нужна только по отдельному требованию преподавателя.

Для практической работы 3 покажите контрольный пример и собственные запуски, проверяющие каждую ветвь. Для практической работы 4 покажите контрольный пример, запуск с минимально допустимым числом элементов и проверку границы условия. Если ввод заканчивается специальным значением, проверьте также завершение ввода до первой записи.

Скриншоты должны содержать исходные данные и итог программы. Если результат отличается от ручного расчёта, сначала проверьте единицы измерения, типы операндов при делении и границы сравнений. Изменённый после проверки код нужно запустить повторно.

На защите работы необходимо объяснить условия выбора, назначение переменных и завершение цикла. Нужно уметь проследить выполнение программы для небольшого набора данных и показать, какая ветвь или итерация обрабатывает каждое значение.

Контрольные вопросы

1. Чем операция присваивания `=` отличается от сравнения `==`? Какой тип имеет результат сравнения?
2. Как записать условие принадлежности числа закрытому интервалу от `a` до `b`? Как проверить выход за его пределы?
3. В каком случае правая часть выражения с `&&` или `||` не вычисляется? Как это использовать перед делением?
4. Чем несколько отдельных `if` отличаются от цепочки `if – else if – else`?
5. Почему в цепочке проверок имеет значение порядок условий? Приведите пример с числовыми границами.
6. Когда удобнее применять `switch`? Для чего нужны `default` и `break`?
7. В каких случаях тело `for` или `while` может не выполниться ни разу? Чем в этом отношении отличается `do while`?
8. Какие действия выполняются в заголовке `for` и в каком порядке?
9. Чем счётчик количества отличается от накопителя суммы? Где нужно задавать их начальные значения?
10. Почему нельзя вычислять среднее отобранных значений, пока не проверено их количество?
11. Как выбрать начальное значение минимума и максимума для серии, которая может содержать отрицательные числа?
12. Чем различаются `break`, `continue` и `return`? Что произойдёт с условием завершения цикла после `continue`?

Справочные материалы Microsoft Learn: [операторы выбора](#); [операторы итерации](#); [логические операции](#).